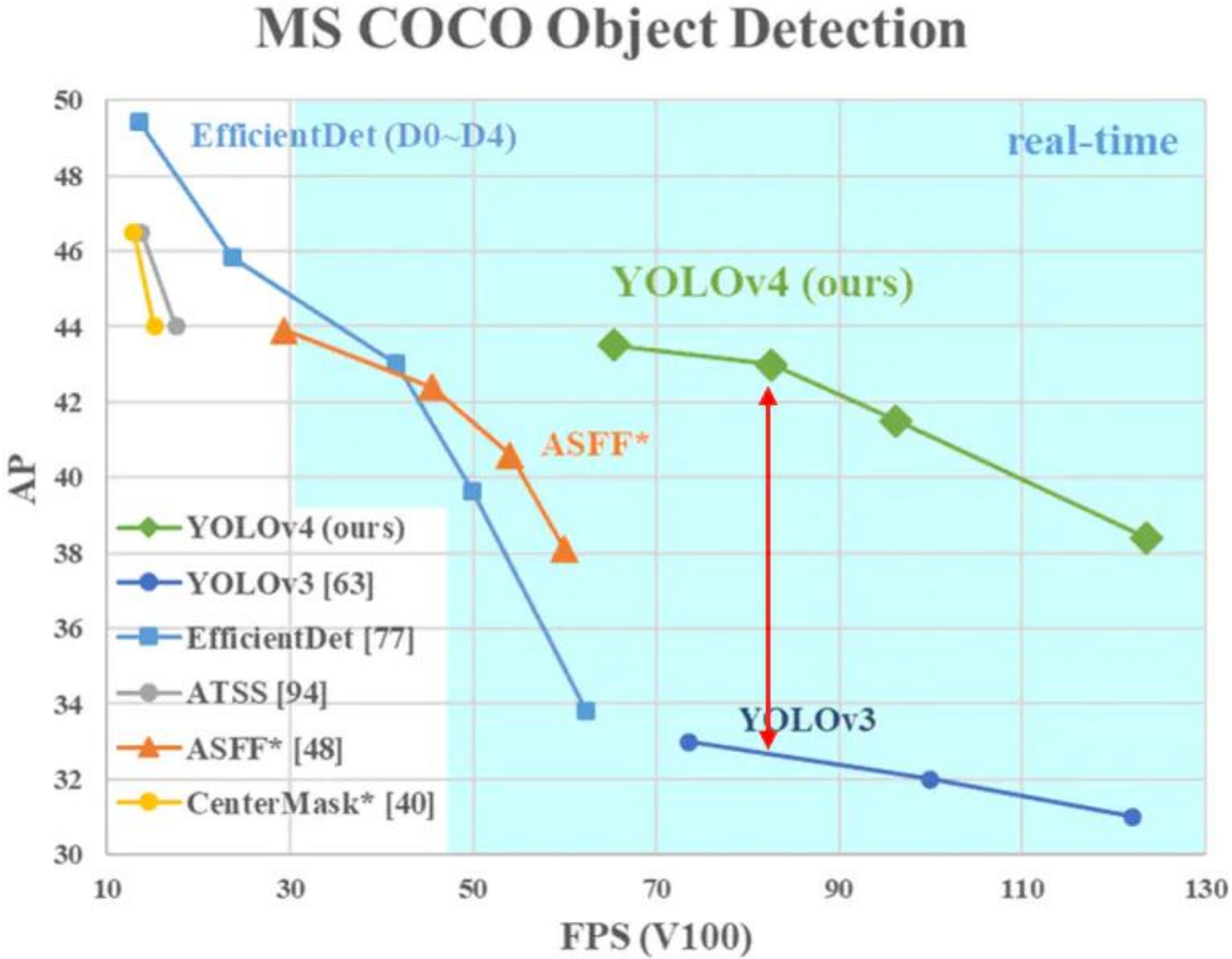
CV项目
YOLO v4

YOLO v4


MS COCO Object Detection
EfficientDet (D0~D4)
real-time
YOLOv4 (ours)
ASFF*
YOLOv3
YOLOv4 (ours)
YOLOv3 [63]
EfficientDet [77]
ATSS [94]
ASFF* [48]
CenterMask* [40]
AP
FPS (V100)
50
48
46
44
42
40
38
36
34
32
30
10
30
50
70
90
110
130

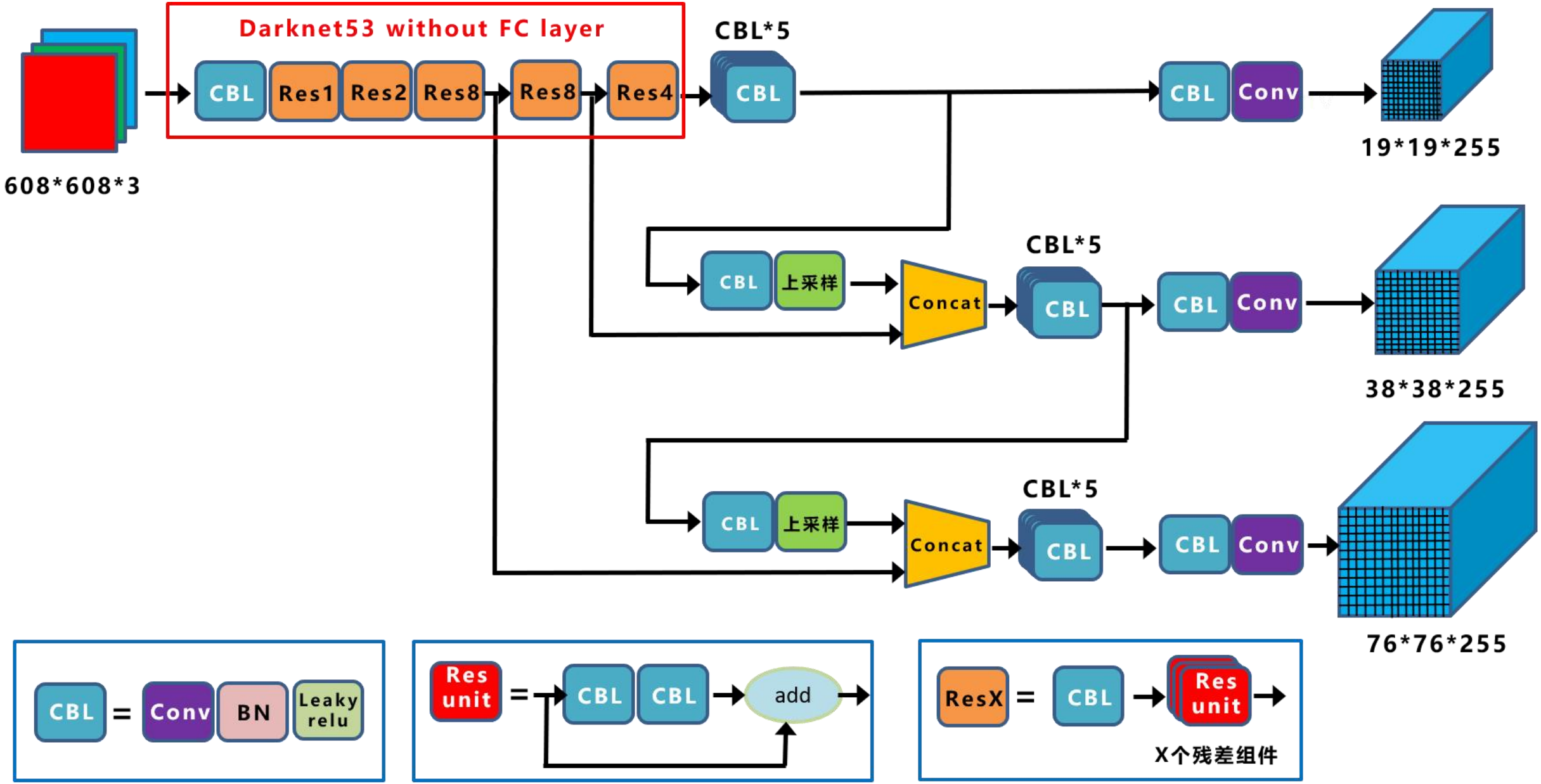
YOLO v4

- 论文：Yolov4: Optimal Speed and Accuracy of Object Detection
- 论文地址： https://arxiv.org/pdf/2004.10934.pdf

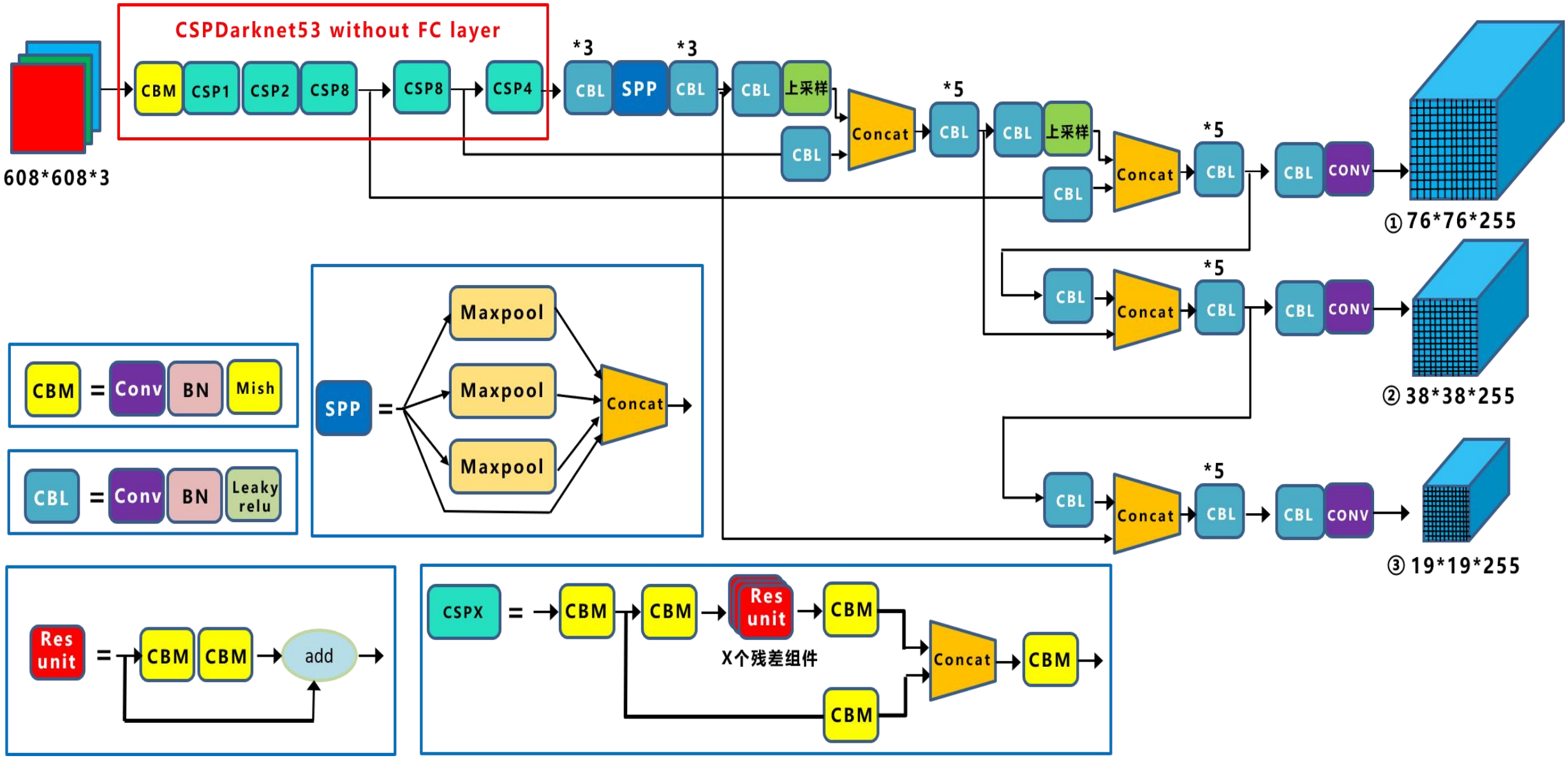
YOLO v4

- YOLO v4中的改进点：
  - 输入端：数据增强(**Mosaic Augmentation**、MixUp、CutMix、Bluring、Label Smoothing等)；
  - BackBone主干网络: **CSPDarkNet-53、Mish激活函数、DropBlock**等；
  - Neck：**PAN**(Path Aggregation Network)、SPP等
  - Prediction：微调边框坐标转换公式、更换边框回归损失函数(CIoU_LOSS)、预选框筛选nms(IoU_nms)更换为DIoU_nms等；

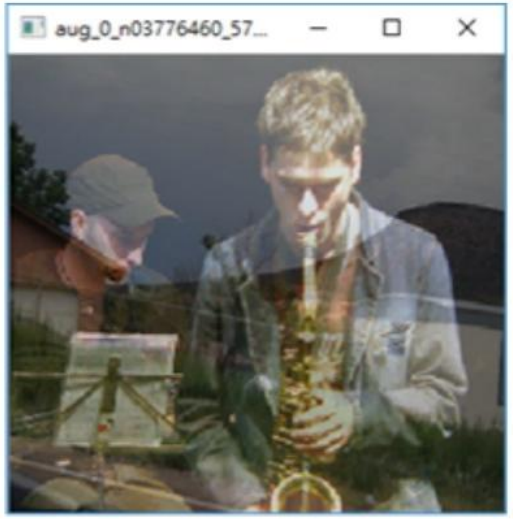
YOLO v3


608*608*3
Darknet53 without FC layer
CBL
Res1
Res2
Res8
Res8
Res4
CBL*5
CBL
CBL
Conv
19*19*255
CBL
上采样
Concat
CBL*5
CBL
CBL
Conv
38*38*255
CBL
上采样
Concat
CBL*5
CBL
CBL
Conv
76*76*255
CBL = Conv BN Leaky relu
Res unit = CBL CBL add
ResX = CBL Res unit
X个残差组件

YOLO v4

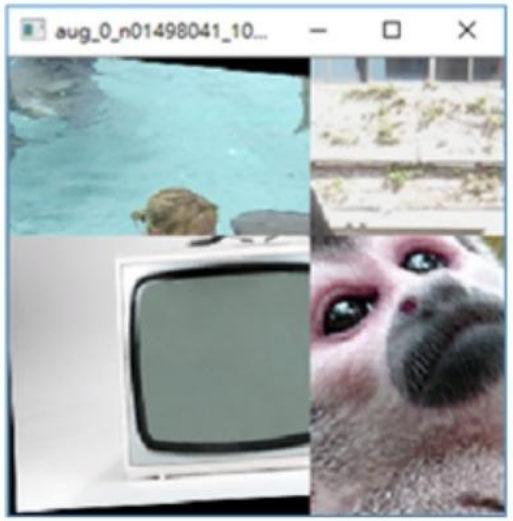

CSPDarknet53 without FC layer
608*608*3
CBM
CSP1
CSP2
CSP8
CSP8
CSP4
*3
CBL
SPP
CBL
*3
CBL
上采样
CBL
Concat
*5
CBL
CBL
上采样
CBL
Concat
*5
CBL
CBL
CONV
①76*76*255
CBL
Concat
*5
CBL
CBL
CONV
②38*38*255
CBL
Concat
*5
CBL
CBL
CONV
③19*19*255
CBM = Conv BN Mish
CBL = Conv BN Leaky relu
SPP = Maxpool Maxpool Maxpool Concat
Res unit = CBM CBM add
CSPX = CBM CBM Res unit CBM X个残差组件 CBM Concat CBM

YOLO v4



(b) MixUp

(c) CutMix

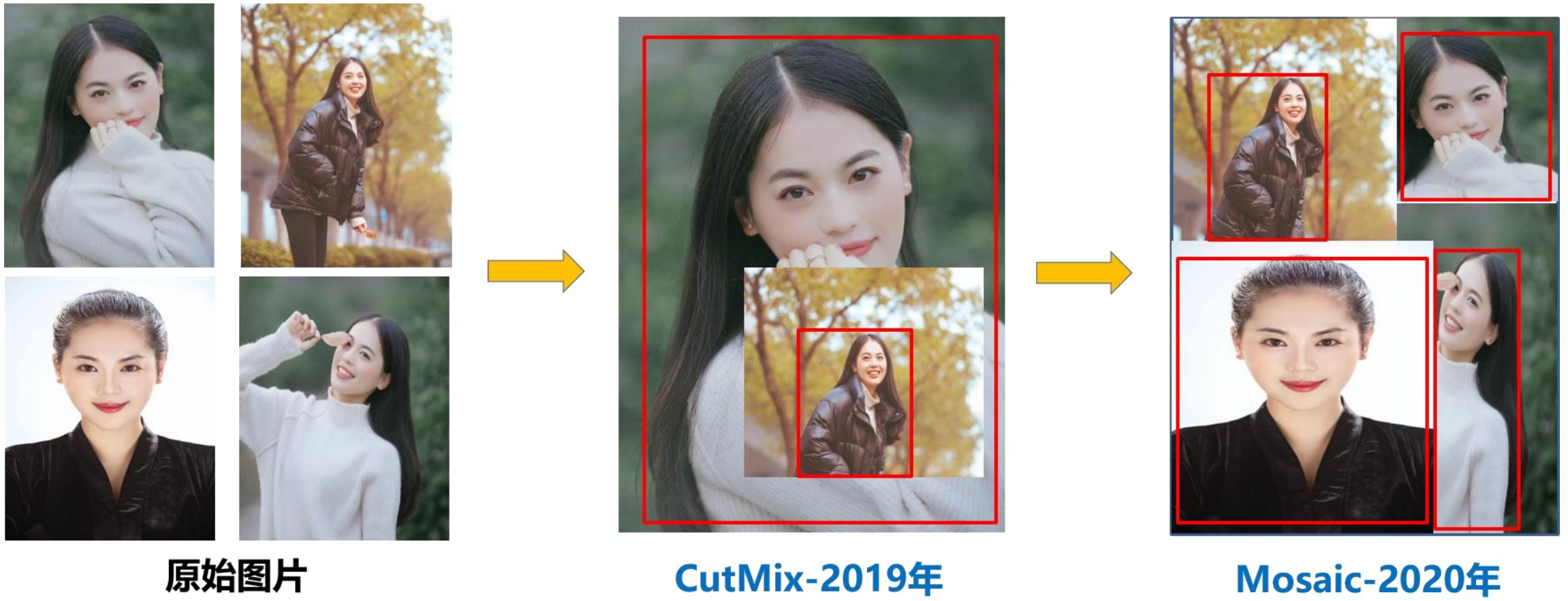


(d) Mosaic

(e) Blur

YOLO v4

- Mosaic是参考CutMix数据增强的方式，但是CutMix只使用两张图像，而Mosaic使用四张图像进行随机缩放、随机剪裁、随机排布的方式进行拼接。



原始图片　　CutMix-2019年　　Mosaic-2020年

YOLO v4

- COCO数据集中，小中大目标分布实际上是不均匀的。

	Min rectangle area	Max rectangle area
Small object	**0*0**	**32*32**
Medium object	32*32	96*96
Large object	96*96	∞*∞

	Small	Mid	Large
Ratio of total boxes(%)	41.4	34.3	24.3
Ratio of images included(%)	52.3	70.7	83.0

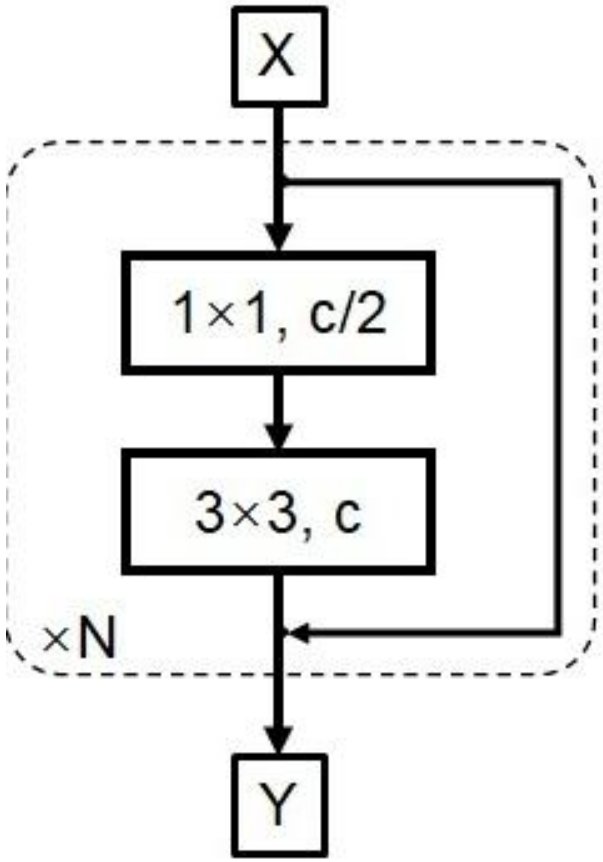
YOLO v4

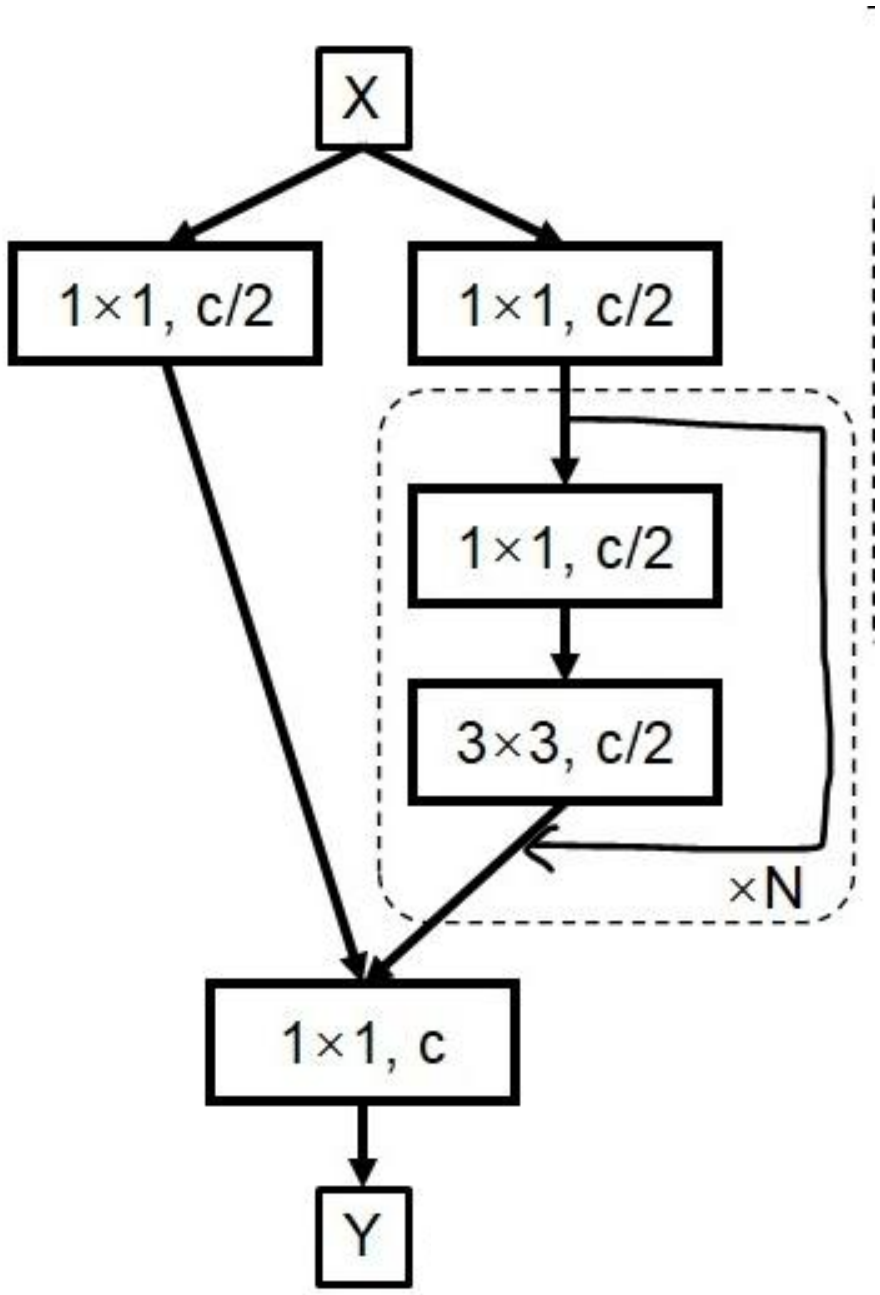
- 采用Mosaic数据增强的主要优点：
  - 丰富数据集
  - 减少GPU/CPU的运算量

YOLO v4

- CSPDarknet53是在YOLO v3的Darknet53的基础上，借鉴了CSPNet(Cross Stage Paritial Network)，主要从网络结构设计的角度解决推理中计算量大的问题。主要思想是：计算量过高的主要问题是由于网络结构中的**梯度信息重复**导致的。所以在CSP结构中，采用将输入层的特征划分为两个部分，一个部分做普通特征提取，另外一个部分直接skip connection即可(类似分类网络结构：ShuffleNet V2)；
- 将Darnet53中的所有激活函数从LeakyRelu更换为Mish激活函数；
- 将Dropout更改为DropBlock结构来缓解过拟合(借鉴cutout数据增强方式)；

YOLO v4

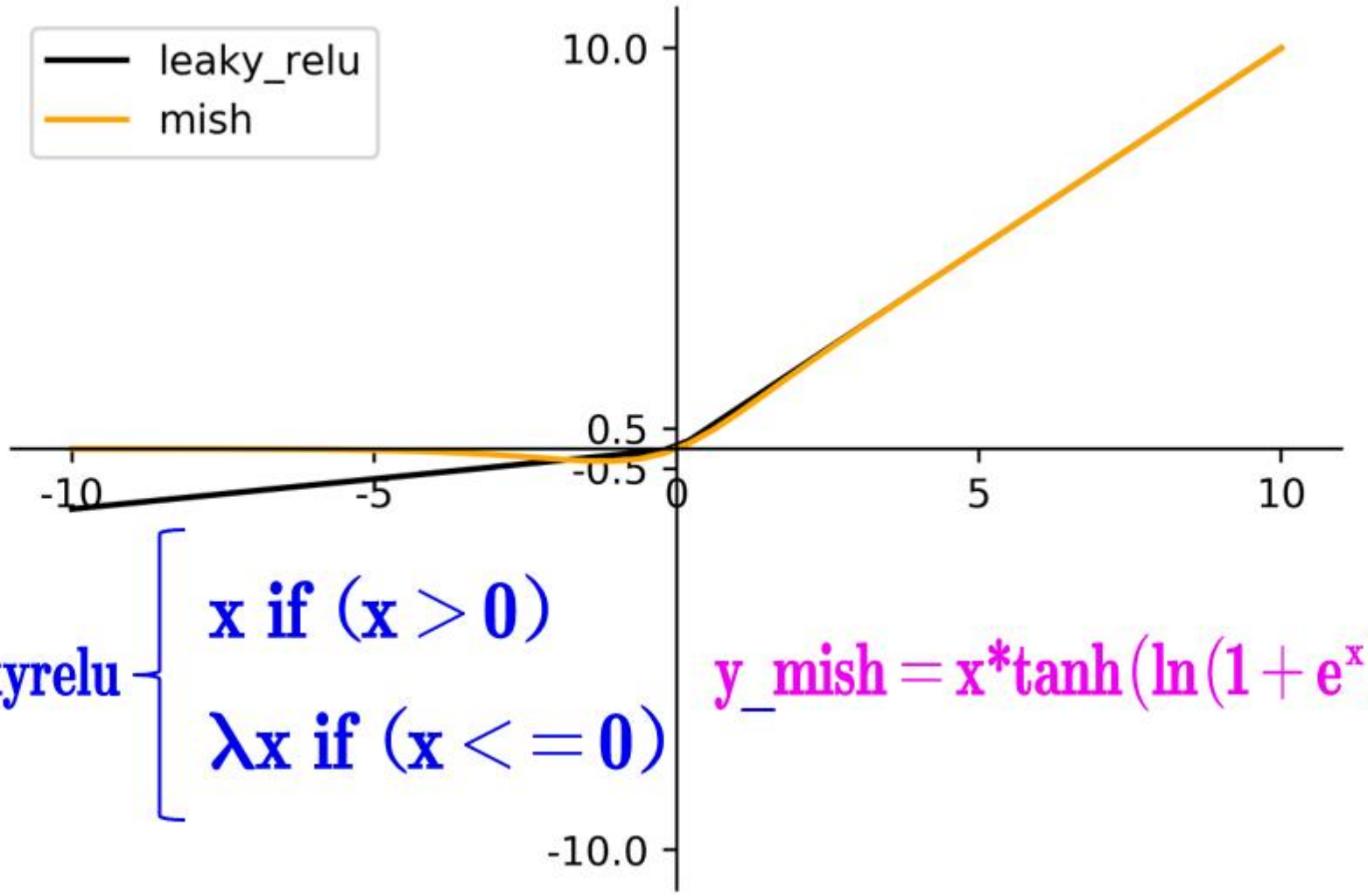




YOLO v4

- 使用CSPDarknet53网络结构的主要优点：
  - 增强CNN的学习能力，使得在轻量化模型的基础上保持准确性；
  - 降低计算瓶颈；
  - 降低内存成本；

YOLO v4



$$y\_leakyrelu \begin{cases} x \text{ if } (x > 0) \\ \lambda x \text{ if } (x <= 0) \end{cases}$$

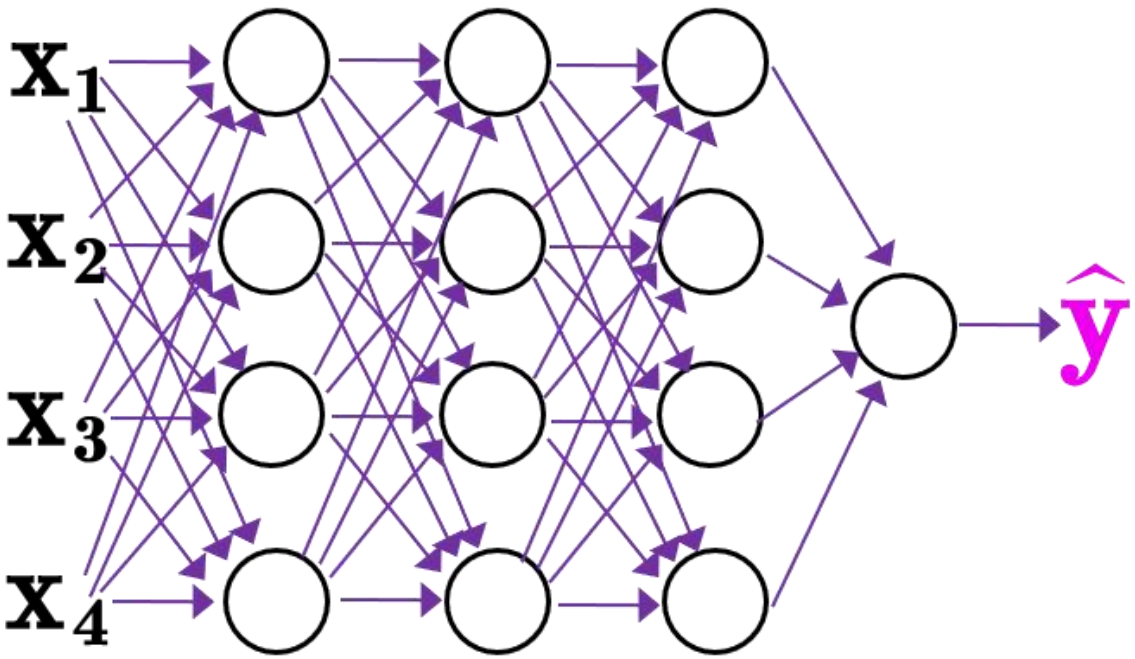
$$y\_mish = x*\tanh\left(\ln\left(1+e^{x}\right)\right)$$

YOLO v4

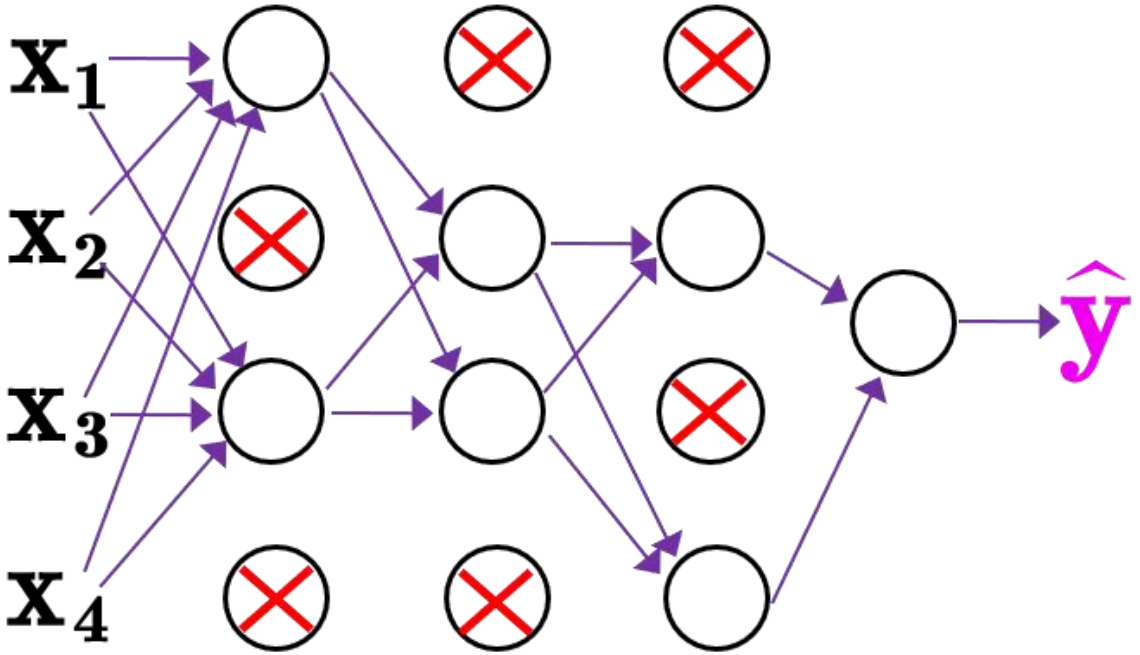
```
class Mish(nn.Module):
    def __init__(self):
        super().__init__()
        print("Mish activation loaded...")

    def forward(self, x):
        x = x * (torch.tanh(F.softplus(x)))
        return x
```

YOLO v4



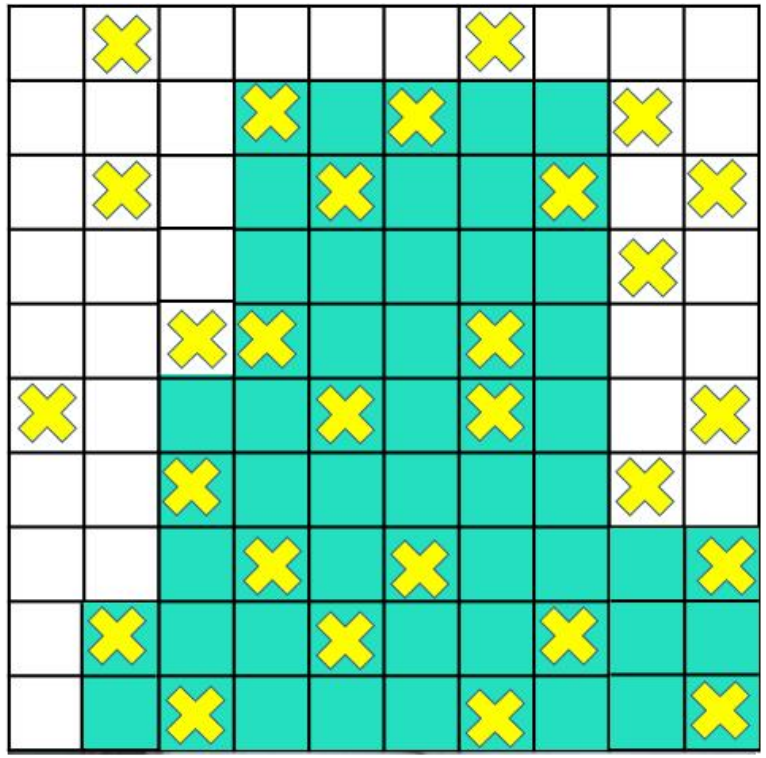
原始网络结构



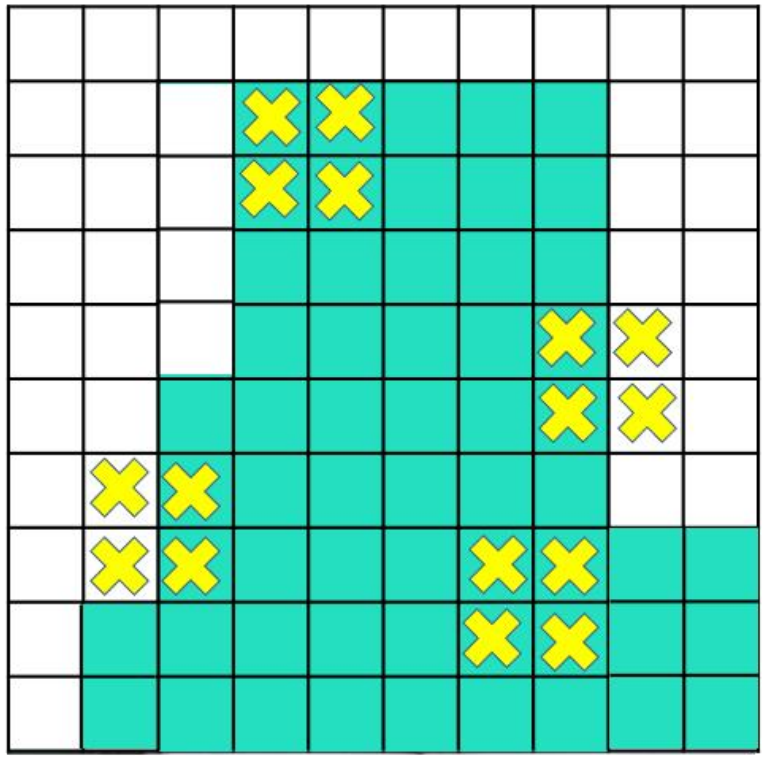
添加dropout后的网络

YOLO v4

原始图片

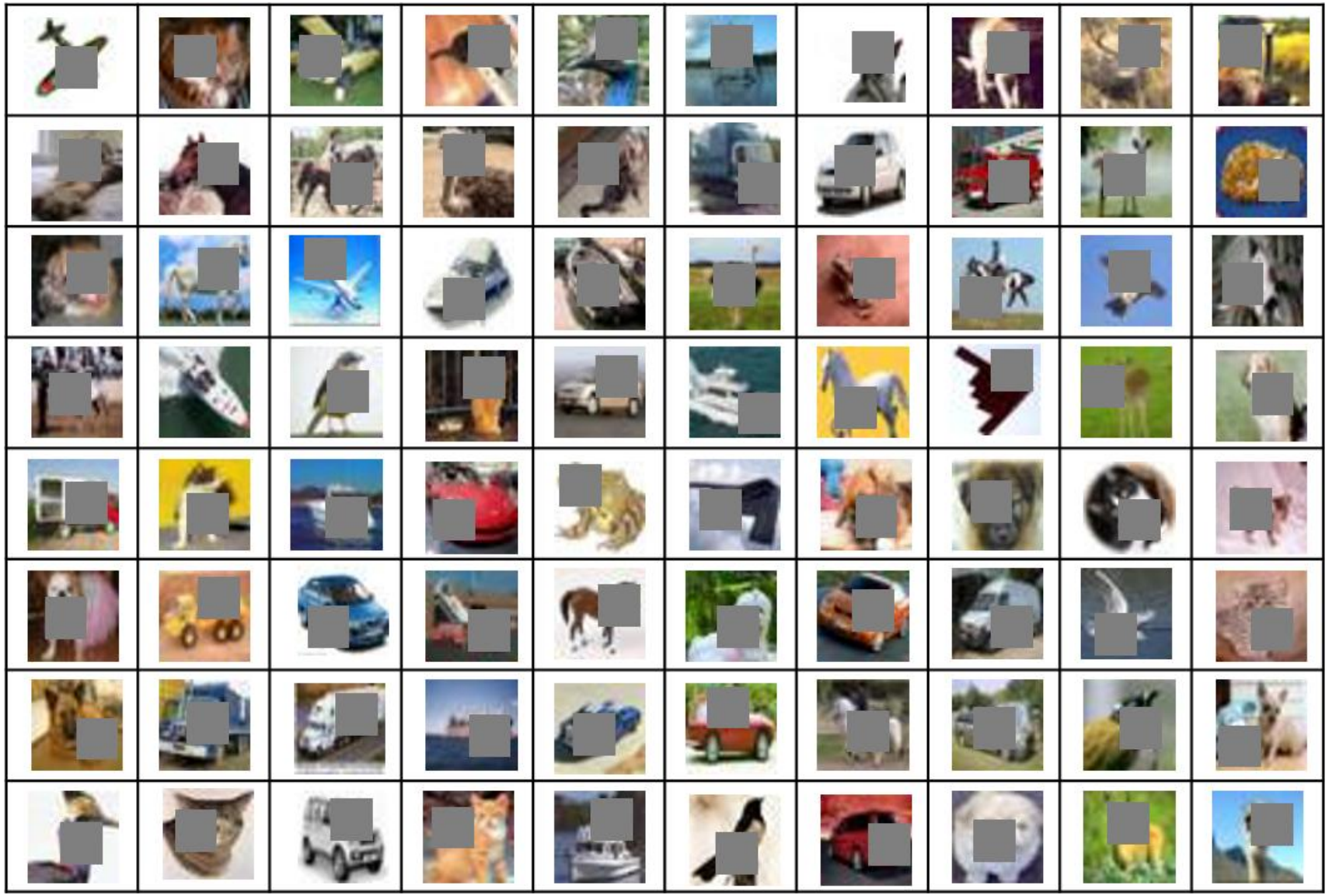


Dropout



Dropblock

YOLO v4



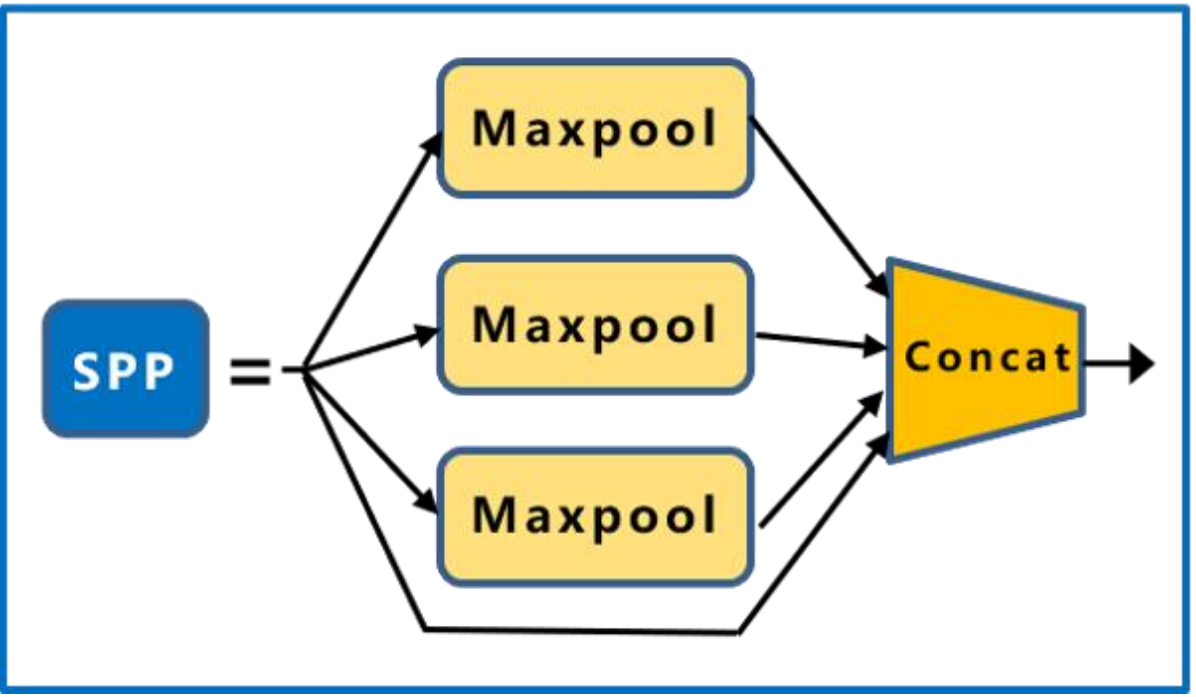
CIFAR-10 上Cutout数据增强

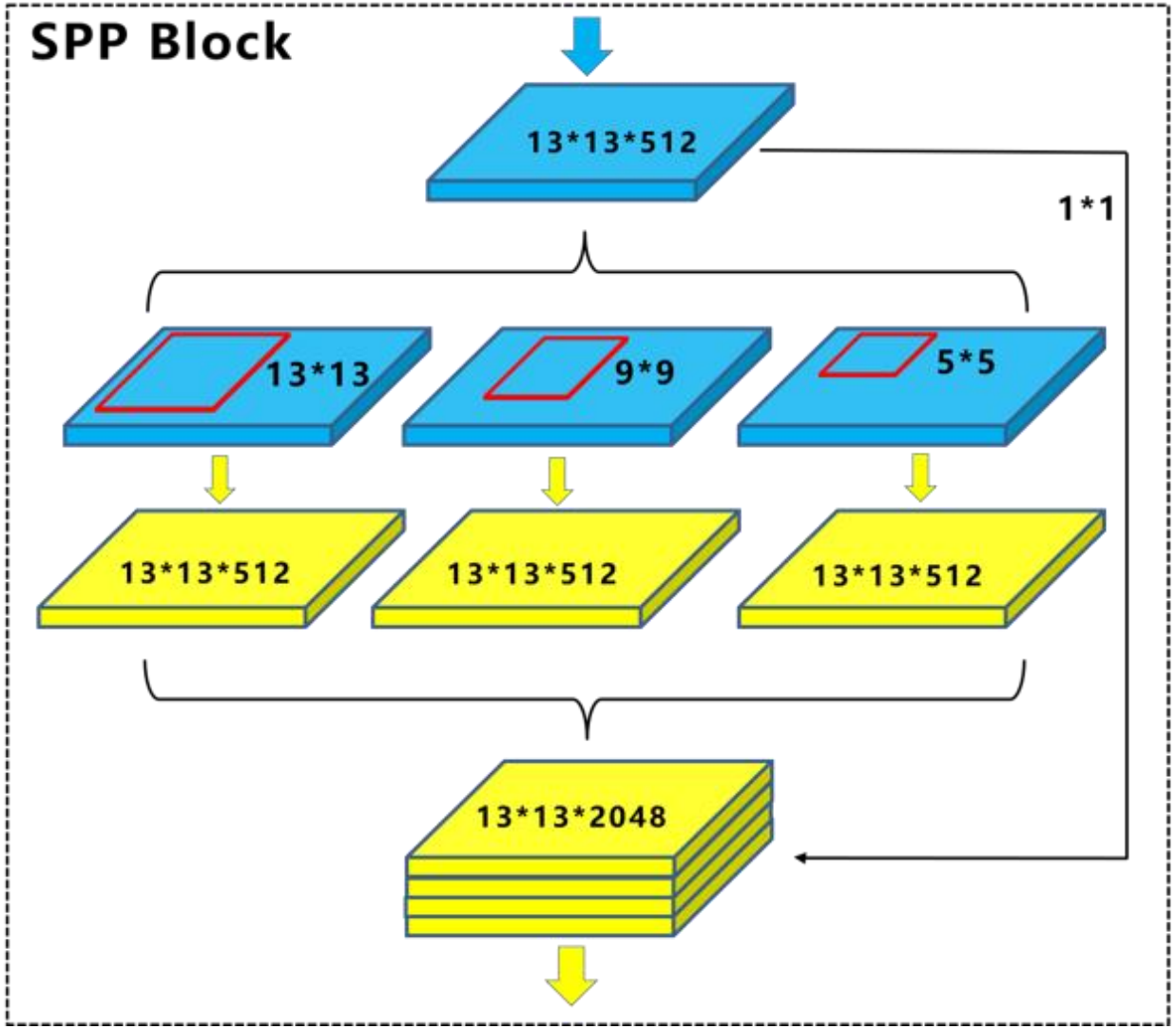
YOLO v4

- Dropblock和Cutout数据增强相比，主要优点如下：
  - Dropblock效果优于Cutout;
  - Cutout仅作用于输入层，而Dropblock可以作用于所有特征图上；
  - Dropblock在训练过程中，可以通过动态修改drop归零的比率让模型效果更加好。

YOLO v4

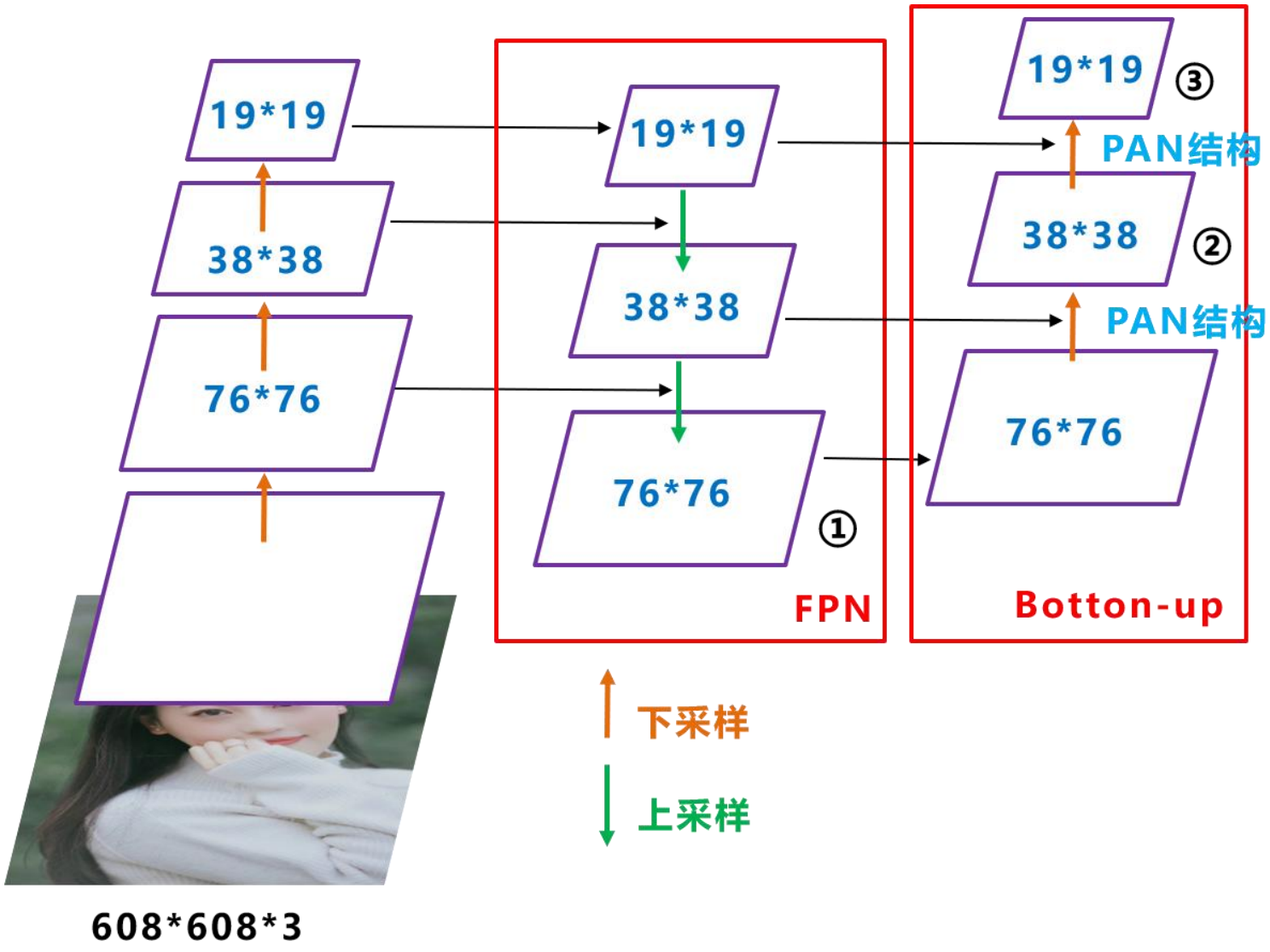
- 使用SPP空间金字塔结构进行特征信息的融合；





YOLO v4

- 借鉴PANet的网络结构优点，在FPN的基础上做一个Botton-up的特征融合。



YOLO v2&v3

- 边框转换存在一个“grid sensitive”问题，针对中心点落在单元格边缘位置的物体不好预测。

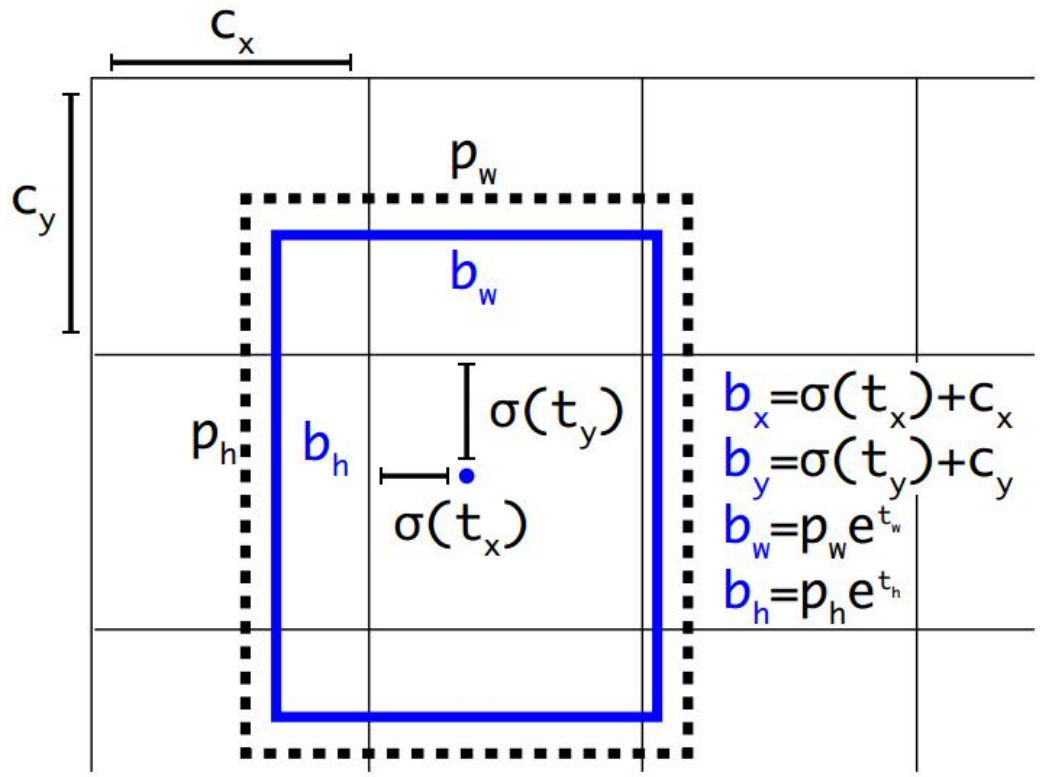


Figure 3: Bounding boxes with dimension priors and location prediction. We predict the width and height of the box as offsets from cluster centroids. We predict the center coordinates of the box relative to the location of filter application using a sigmoid function.

YOLO v4

$$b_x = s * \sigma(t_x) + c_x$$

$$b_y = s * \sigma(t_y) + c_y$$

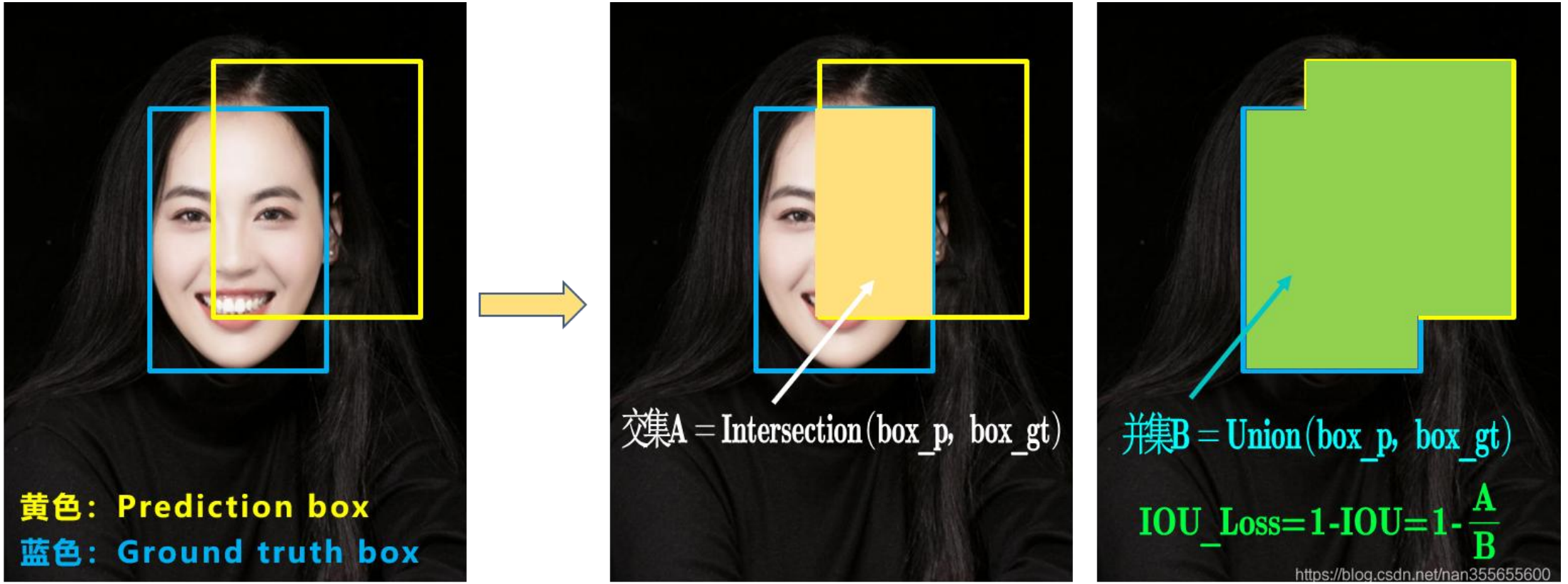
YOLO v4

- 边框回归的损失发展历程: Smooth L1 Loss-> IoU Loss（2016）-> GIoU Loss（2019）-> DIoU Loss（2020）->CIoU Loss（2020）；
- IoU Loss：主要考虑**检测框和目标框重叠面积**。
- GIoU Loss：在IOU的基础上，解决**边界框不重合**时的问题。
- DIoU Loss：在IOU和GIOU的基础上，考虑**边界框中心点距离**的信息。
- CIoU Loss：在DIOU的基础上，考虑**边界框宽高比的尺度**信息。

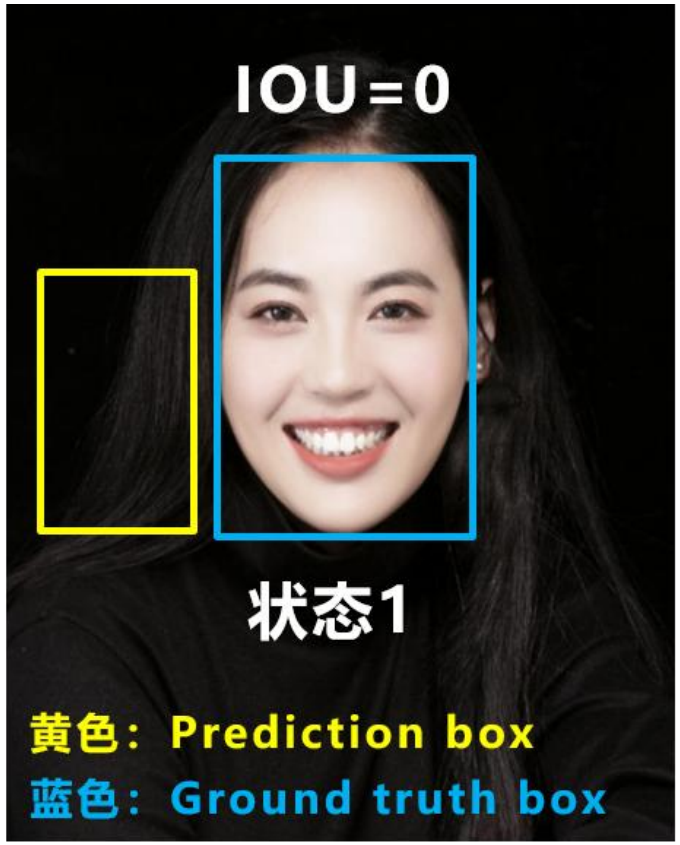
$$CIOU\_Loss=1-CIOU=1-\left(IOU-\frac{Distance\_2^2}{Distance\_C^2}-\frac{\nu^2}{(1-IOU)+\nu}\right)$$

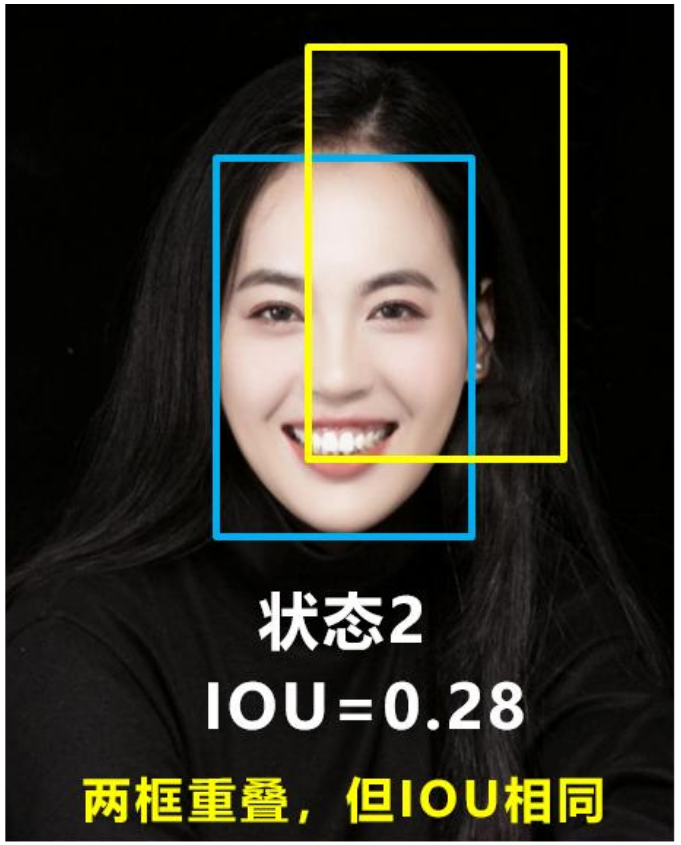
$$\nu=\frac{4}{\pi^2}\left(\arctan\frac{w^{gt}}{h^{gt}}-\arctan\frac{w^{p}}{h^{p}}\right)^2$$

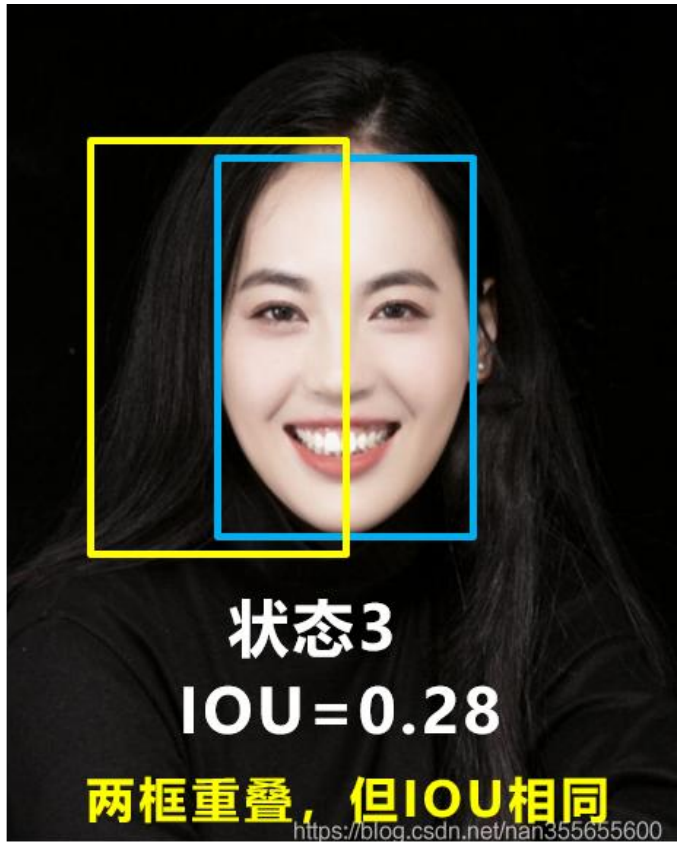
YOLO v4


黄色：Prediction box
蓝色：Ground truth box
交集A = Intersection(box_p, box_gt)
并集B = Union(box_p, box_gt)
IOU_Loss=1-IOU=1-A/B
https://blog.csdn.net/nan355655600

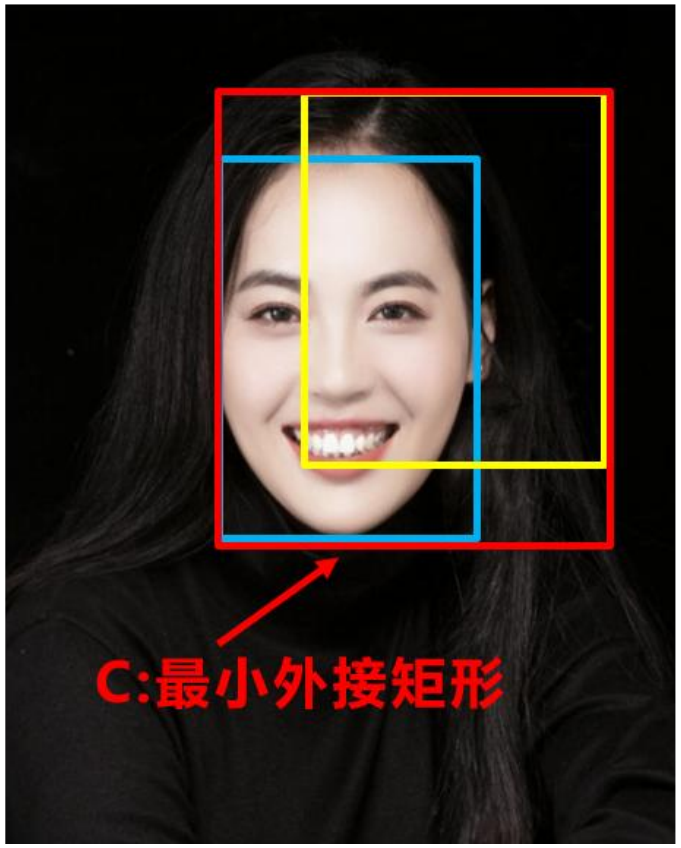
YOLO v4

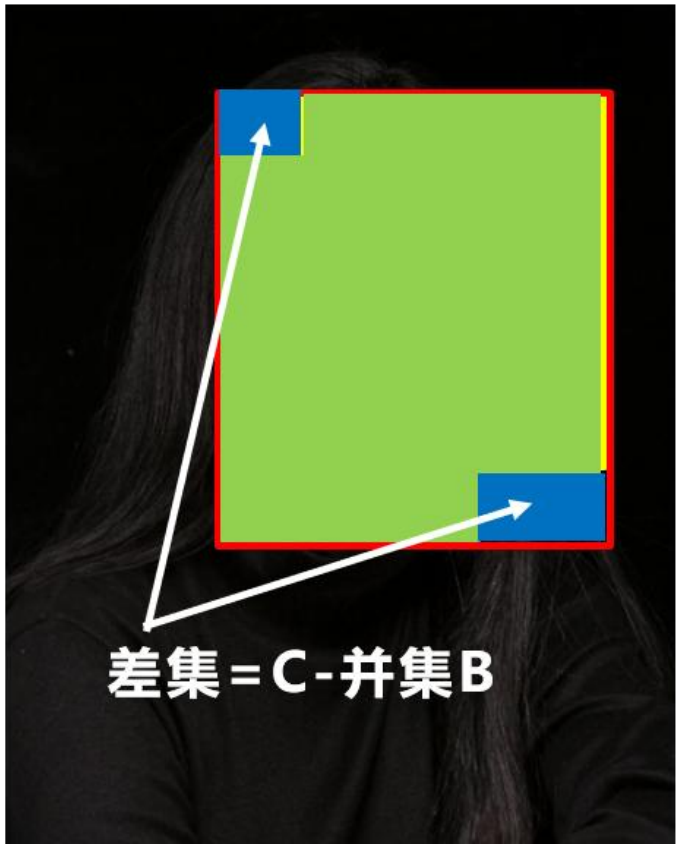

IOU=0
狀态1
黃色：Prediction box
蓝色：Ground truth box

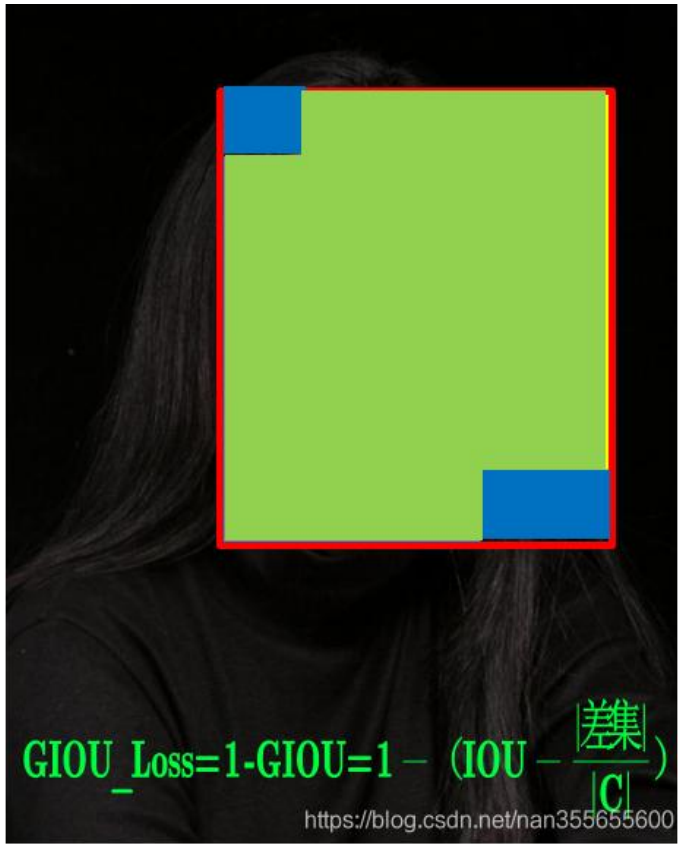

狀态2
IOU=0.28
两框重疊，但IOU相同


狀态3
IOU=0.28
两框重疊，但IOU相同
https://blog.csdn.net/nan355655600

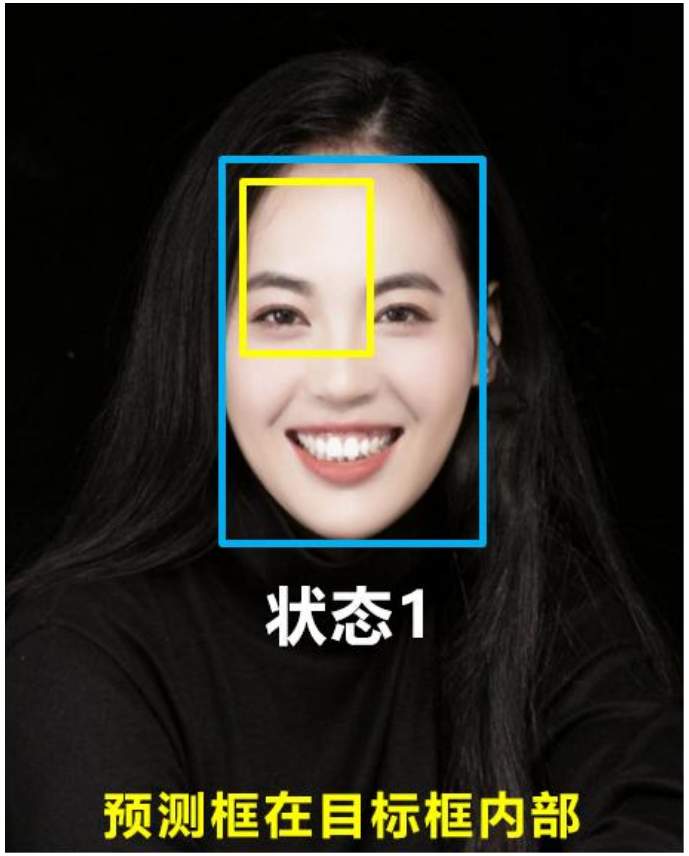
YOLO v4

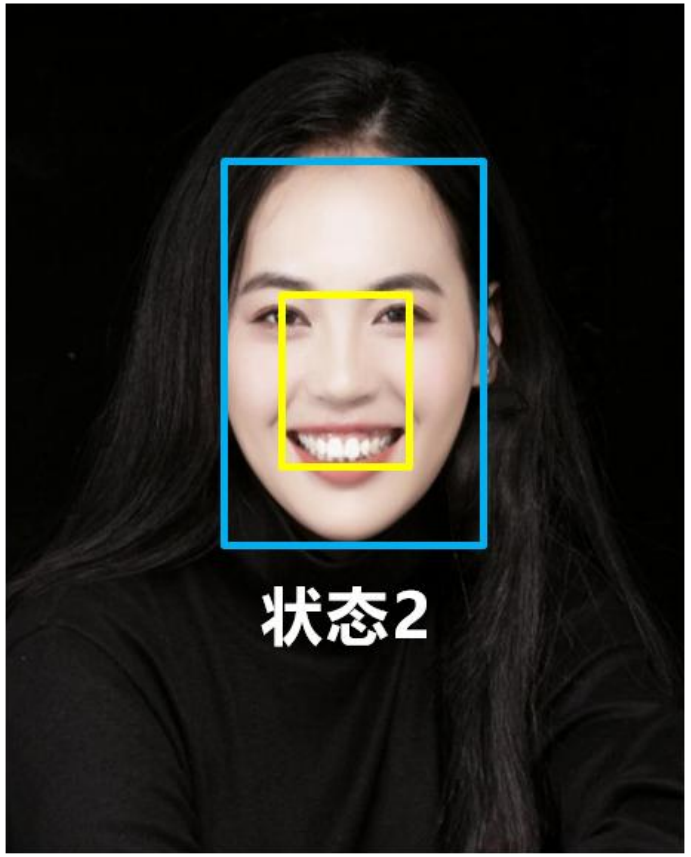

C:最小外接矩形

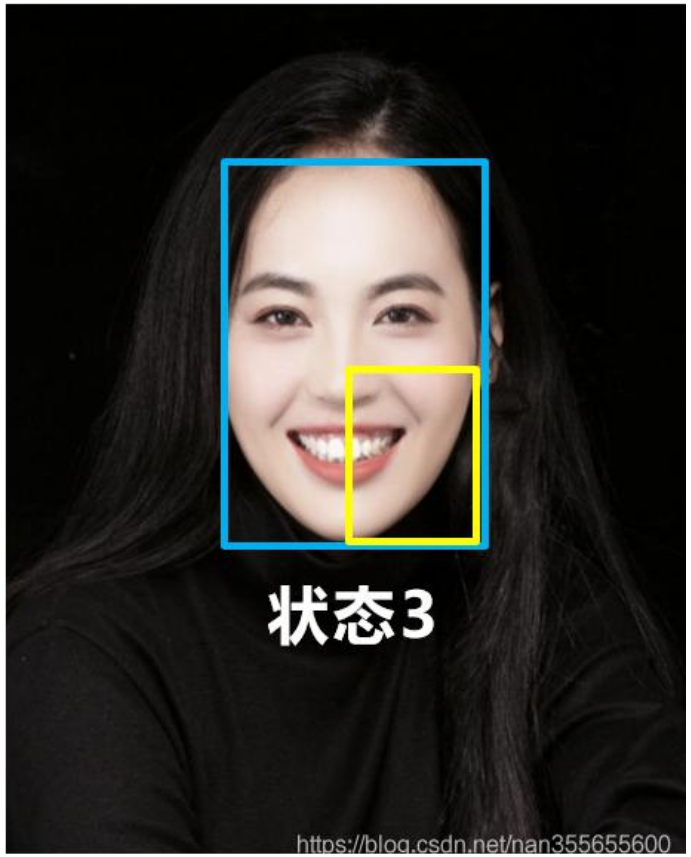

差集=C-并集B


GIOU_Loss=1-GIOU=1-(IOU-差集/|C|)
https://blog.csdn.net/nan355655600

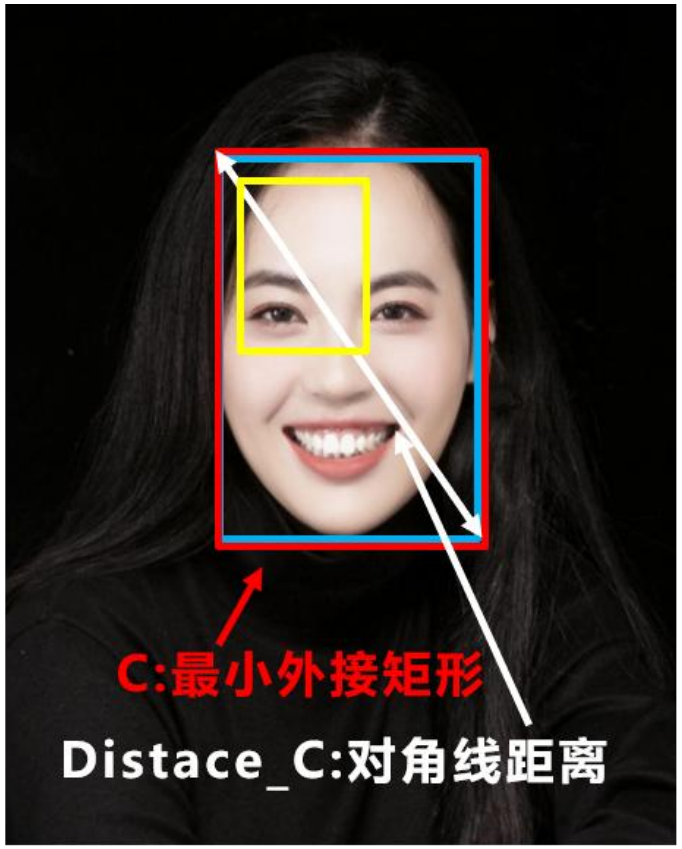
YOLO v4

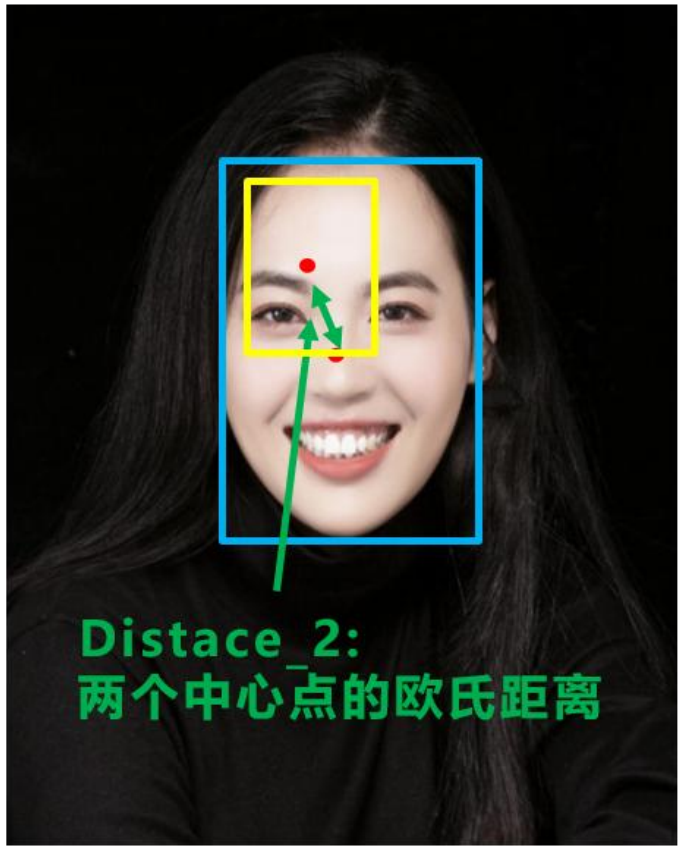


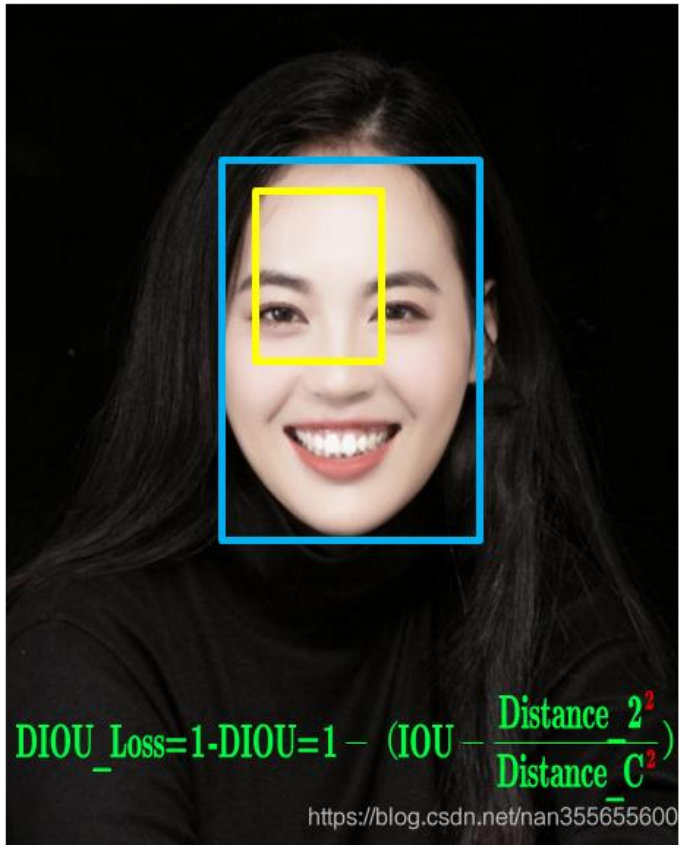




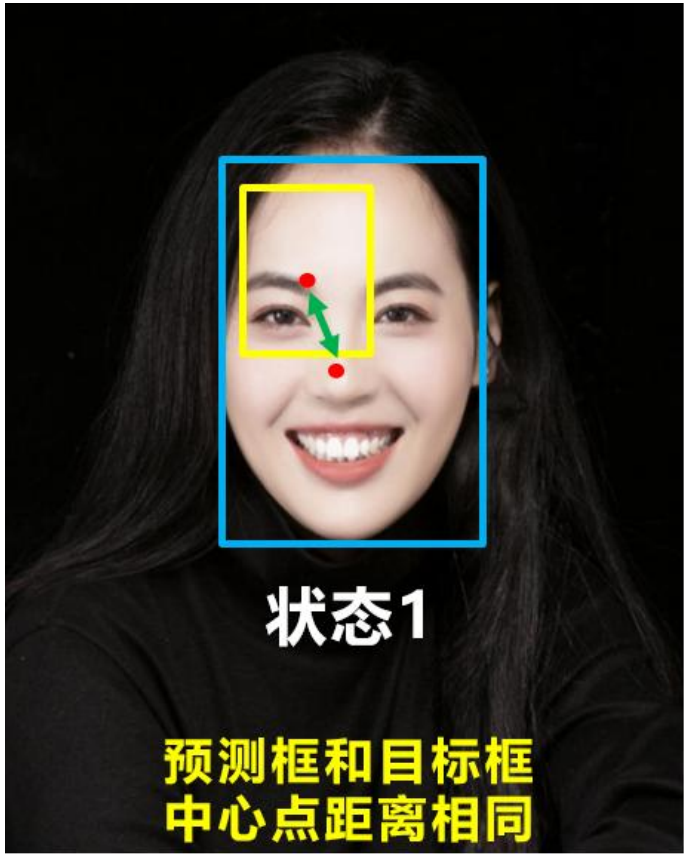
YOLO v4

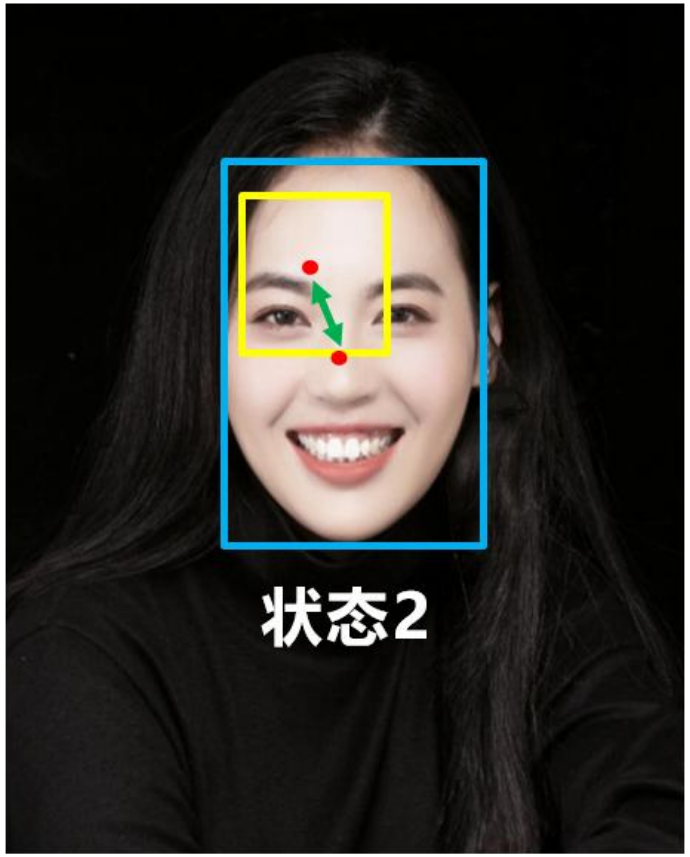

C:最小外接矩形
Distace_C:对角线距离

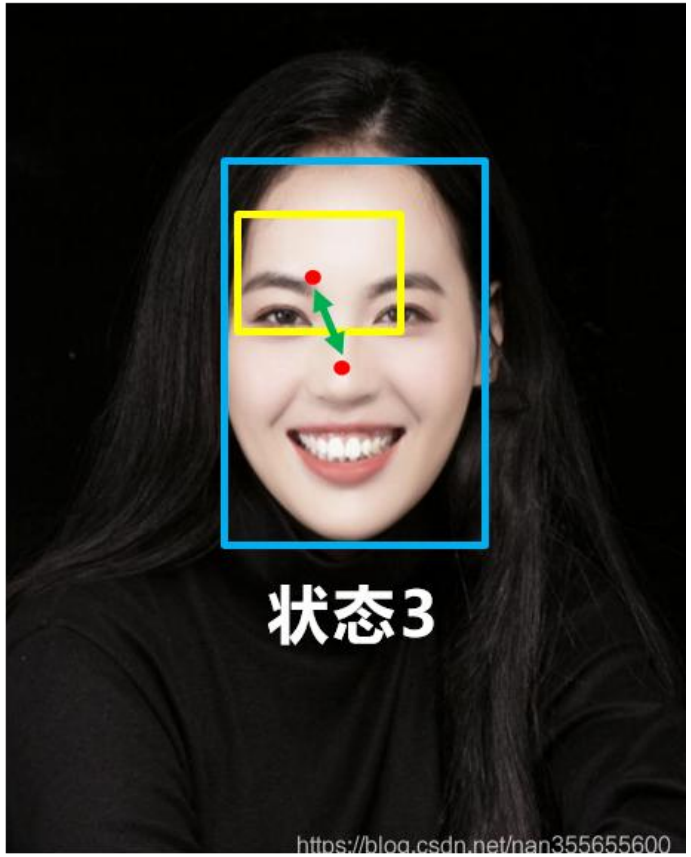

Distace_2:
两个中心点的欧氏距离


DIOU_Loss=1-DIOU=1-(IOU-Distance_2²/Distance_C²)
https://blog.csdn.net/nan355655600

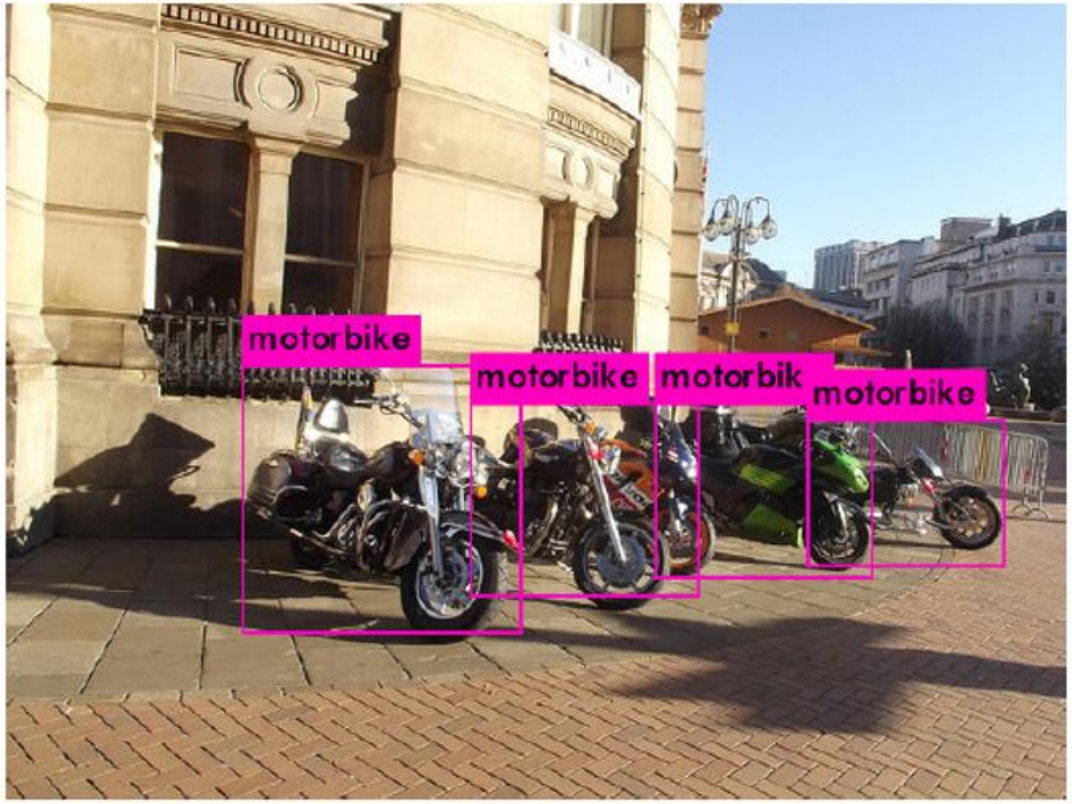
YOLO v4


状态1
预测框和目标框
中心点距离相同

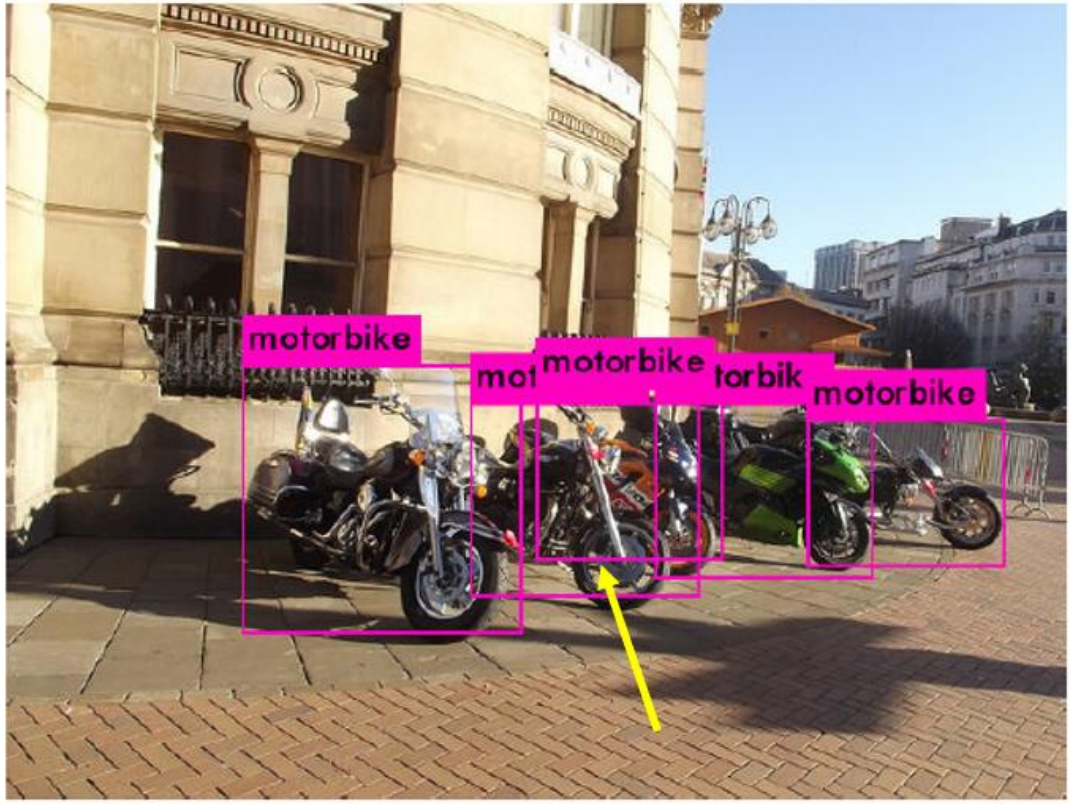

状态2


状态3
https://blog.csdn.net/nan355655600

YOLO v4



CIOU_Loss+NMS



CIOU_Loss+DIOU_nms

THANKS!